

MALLA REDDY UNIVERSITY

Department of Computer Science and Engineering (Cyber Security)

ETHICAL HACKING LABORATORY RECORD

Course Code: MR23-1CS0432

EXPERIMENT 07

Password Security and Authentication Testing

Experiment Number	07 — Password Security and Authentication Testing
Platform	Python 3.14.6 on Windows 11 / Kali Linux (Python 3.10+)
Application	Local educational authentication engine (scripts/app.py, scripts/auth_engine.py)
Dependencies	Python Standard Library only (hashlib, secrets, hmac, math, time, unittest)
Date of Execution	September 22, 2026 (UTC+05:30) — Live execution on this system

1. Aim

To analyse modern password-authentication mechanisms — cryptographic hashing, salting, entropy measurement, and account-lockout rate limiting — using a purpose-built local Python authentication engine (scripts/app.py and scripts/auth_engine.py), and to compare insecure legacy password storage (unsalted MD5) against modern secure design (PBKDF2-HMAC-SHA256 with CSPRNG salts) through live execution and automated unit testing.

2. Objectives

- Compare raw unsalted MD5 hashing against salted PBKDF2-HMAC-SHA256 hashing and demonstrate their practical security difference.
- Observe how a cryptographically secure salt (CSPRNG, 16 bytes) causes two runs on an identical password to produce completely different stored hashes.
- Measure password entropy using the formula $E = L \times \log_2(R)$ and verify the entropy calculation implementation.
- Trigger and observe the account lockout mechanism after three consecutive failed login attempts — demonstrating rate-limiting defence against online brute-force attacks.
- Execute the automated unit test suite (4 tests) using Python unittest to verify each security control programmatically.
- Document real execution evidence — exact terminal output, generated salts, hashes, and test results — to build authenticated submission evidence.

3. Laboratory Environment & Requirements

Execution Platform	Windows 11, Python 3.14.6 (also compatible with Kali Linux Python 3.10+)
Target Application	Local educational authentication engine: scripts/app.py + scripts/auth_engine.py
External Dependencies	None — uses Python Standard Library exclusively (hashlib, secrets, hmac, math, time, unittest)
Test Framework	Python built-in unittest module — no additional packages required
Working Directory	experiment-07-password-authentication/ (or Ethical-Hacking-Experiments-2-and-7/Experiment-07/)

4. Theoretical Background

4.1 Evolution of Password Storage Security

Password security has evolved through four distinct eras. In the 1970s–1990s, passwords were stored in plaintext or using fast cryptographic hash functions such as MD5 and SHA-1. The fundamental flaw of fast hashes is that modern GPU hardware can compute billions of MD5 hashes per second, making offline dictionary and brute-force attacks trivially fast against stolen password databases.

The second era introduced salted hashes — combining a per-user random value (salt) with the password before hashing. While this defeats precomputed rainbow-table attacks, salted fast hashes remain vulnerable to brute-force GPU attacks.

Modern best practice uses Key Derivation Functions (KDFs) — PBKDF2, bcrypt, and Argon2id — which are deliberately computationally slow and configurable. A legitimate system only verifies one password per login; an attacker brute-forcing a stolen database must compute millions or billions of hash iterations, making attacks computationally prohibitive.

4.2 Hashing Algorithm Comparison Matrix

Algorithm	Type	Salt	Speed	GPU Resistance	Status
MD5	Fast Hash	✗ None	Giga/sec	✗ Broken	🚫 DEPRECATED
SHA-256	Fast Hash	Optional	Giga/sec	✗ GPU clusters	🚫 INSUFFICIENT
PBKDF2	KDF	✓ Mandatory	Slow	⚠ Moderate	✓ NIST APPROVED
bcrypt	KDF	✓ Mandatory	Very Slow	✓ High	✓ RECOMMENDED
Argon2id	Mem-Hard	✓ Mandatory	Configurable	✓ Excellent	★ BEST PRACTICE

4.3 Cryptographic Salt and CSPRNG

A cryptographic salt is a unique, unpredictable sequence of random bytes combined with a password before hashing. The salt must be generated by a Cryptographically Secure Pseudo-Random Number Generator (CSPRNG) — in Python, the `secrets` module (`secrets.token_bytes(16)`) rather than the standard `random` module, which is not cryptographically secure.

The salt serves two critical purposes: (1) It defeats precomputed rainbow-table attacks, because each unique salt produces a unique hash even for a common password. (2) It ensures that two users who choose the same password produce entirely different stored hash values, so a single cracked hash reveals nothing about other accounts.

The minimum recommended salt length is 16 bytes (128 bits). The salt is stored in plaintext alongside the hash — this is intentional and safe, because the salt's value is unpredictability at generation time, not secrecy thereafter.

4.4 Password Entropy

Password entropy quantifies the randomness of a password in bits, calculated as:

$$E = L \times \log_2(R)$$

where L = password length (characters), R = character set size (e.g., 26 lowercase + 26 uppercase + 10 digits + 32 symbols = 94 possible characters). A common minimum policy threshold is 50 bits of entropy. The password "LaboratoryPassword2026!" uses all four character classes (L=22, R=94), yielding $E = 22 \times \log_2(94) \approx 22 \times 6.554 \approx 144.19$ bits — well above the minimum threshold.

4.5 Account Lockout and Rate Limiting

Online brute-force attacks attempt many password guesses against a live authentication endpoint. Account lockout mechanisms track consecutive failed login attempts per account and temporarily suspend authentication after a threshold (e.g., 3–5 failures within a window), introducing artificial delays or full lockouts that throttle automated attacks to negligible speeds.

Constant-time hash comparison (Python's `hmac.compare_digest()`) prevents timing attacks — a side-channel attack that measures subtle variations in string comparison duration to infer how many initial characters of the hash match, allowing incremental cracking without full hash inversion.

5. Step-by-Step Practical Procedure

Both steps below were actually executed on this Windows 11 system on September 22, 2026. All outputs below are real — captured from live execution of the supplied scripts. No output has been fabricated or simulated.

Step 1 — Execute the Automated Unit Test Suite

Objective	Verify that password-entropy calculation, CSPRNG-based salting, user registration/authentication, and account-lockout rate limiting each behave as designed — using automated, repeatable unit tests.
Technical Concept	An automated unit-test suite exercises each security control in isolation, providing code-level verification independent of any interactive terminal session. Four distinct tests cover the four core controls implemented in <code>auth_engine.py</code> .

Command Executed:

```
python -m unittest discover -s scripts/tests -p "test_*.py" -v
```

Command Flag Breakdown:

- **python -m unittest discover**: Activates Python's built-in unittest discovery mode — automatically finds test files without specifying them manually.
- **-s scripts/tests**: Sets the start directory for test discovery to the scripts/tests/ subdirectory.
- **-p "test_*.py"**: Pattern matching — discovers all Python files whose names begin with test_.
- **-v**: Verbose mode — prints each individual test name and its pass/fail result as it runs.

Test Name	What It Verifies	Expected Assertion
test_entropy_calculation	Entropy of "abc" < entropy of "P@ssw0rd2026!#\$" — low vs high complexity strings.	assertGreater(high_entropy, low_entropy)
test_salting_uniqueness	Two PBKDF2 hashes of the same password with CSPRNG salts produce different salt and hash values.	assertNotEqual(salt1, salt2) and assertNotEqual(hash1, hash2)
test_user_registration_and_authentication	User can register and then successfully authenticate with correct password.	assertTrue(success) and assertIn("successful", msg)
test_account_lockout	Three consecutive wrong passwords trigger lockout; correct password is then blocked during lockout window.	assertFalse(success) and assertIn("locked", msg)

Actual Terminal Output (Live Execution — September 22, 2026):

```
$ python -m unittest discover -s scripts/tests -p "test_*.py" -v
test_account_lockout (test_auth.TestAuthEngine.test_account_lockout) ... ok
test_entropy_calculation (test_auth.TestAuthEngine.test_entropy_calculation) ... ok
test_salting_uniqueness (test_auth.TestAuthEngine.test_salting_uniqueness) ... ok
test_user_registration_and_authentication
(test_auth.TestAuthEngine.test_user_registration_and_authentication) ... ok

-----
Ran 4 tests in 0.161s

OK
```

- All 4 unit tests passed: test_account_lockout ... ok, test_entropy_calculation ... ok, test_salting_uniqueness ... ok, test_user_registration_and_authentication ... ok.

- Total execution time: 0.161 seconds — fast enough for integration into a CI/CD pipeline.
- Final summary line: "Ran 4 tests in 0.161s" followed by "OK" — confirms zero test failures.
- The test_salting_uniqueness result proves that two sequential PBKDF2 hashes of the same password with CSPRNG salts always produce different output — the core security property of salting.
- The test_account_lockout result confirms the lockout state persists and blocks even the correct password during the lockout window.

✓ **RESULT:** All 4 automated unit tests PASSED in 0.161 seconds. Each security control — entropy calculation, CSPRNG salting uniqueness, user authentication, and account lockout — verified correct programmatically.

```

PS D:\Ethical Hacking Experiments\Ethical-Hacking-Experiments-2-and-7\Experiment-07> python -m unittest discover -s scripts/tests -p "test_*.py"
test_account_lockout (test_auth.TestAuthEngine.test_account_lockout) ... ok
test_entropy_calculation (test_auth.TestAuthEngine.test_entropy_calculation) ... ok
test_salting_uniqueness (test_auth.TestAuthEngine.test_salting_uniqueness) ... ok
test_user_registration_and_authentication (test_auth.TestAuthEngine.test_user_registration_and_authentication) ... ok

-----
Ran 4 tests in 0.161s

OK

PS D:\Ethical Hacking Experiments\Ethical-Hacking-Experiments-2-and-7\Experiment-07> _

```

Figure 7.1: Automated unit test discovery and execution (`python -m unittest discover -s scripts/tests -p "test_*.py" -v`) showing all 4 tests passing: "Ran 4 tests in 0.161s — OK".

Step 2 — Execute the Interactive CLI Authentication Testing Engine

Objective	Interactively demonstrate: (1) unsalted MD5 vs salted PBKDF2 hash comparison, (2) CSPRNG salt uniqueness producing different hashes for identical passwords, (3) entropy measurement, (4) account lockout rate-limiting triggered after 3 failed login attempts.
Technical Concept	Running the CLI tool interactively, rather than only through automated tests, allows direct observation of the cryptographic outputs — a student can see two identical passwords produce entirely different stored hashes once a unique salt is introduced per execution run.

Command Executed:

```
python scripts/app.py
```

Actual Terminal Output (Live Execution — September 22, 2026):

```
$ python scripts/app.py
=====
EXPERIMENT 07: PASSWORD SECURITY & AUTHENTICATION ENGINE
Course: MR23-1CS0432 - Ethical Hacking Laboratory (MRU)
=====

[1] Password Hashing & Salt Demonstration
Raw Input Password: LaboratoryPassword2026!
Password Entropy: 150.76 bits

[INSECURE] Unsalted MD5 Hash: 096b1f381d46137f604347537db76776

[SECURE] PBKDF2 (Run 1):
  Salt 1 (Hex): fcd7d5c65d34c46348a007bdae2a3278
  Hash 1 (Hex): 3529fa3622127a42371c35293515abfb1f8a9e26ba25df453549a5b7084b614d

[SECURE] PBKDF2 (Run 2 - Same Password, New CSPRNG Salt):
  Salt 2 (Hex): aaa275edf37f18f86af01e845afb6fec
  Hash 2 (Hex): 29274da953a6e5ef83bd4ed9626c24c61f1c5768d647cd70aa5df5c6ebc90c0b

[2] Registering Synthetic User 'student_admin'...
Registration Status: SUCCESS

[3] Simulating Failed Login Attacks & Account Lockout Defense:
Attempting '123456' -> [DENIED]: Invalid credentials. Attempts remaining before
lockout: 2
Attempting 'admin123' -> [DENIED]: Invalid credentials. Attempts remaining before
lockout: 1
Attempting 'wrong_pass' -> [DENIED]: Account locked! Exceeded max failed attempts (3).
Attempting 'LaboratoryPassword2026!' -> [DENIED]: Account locked due to multiple failed
attempts. Try again in 9 seconds.

=====
LOCAL PRACTICAL EXECUTION TEST COMPLETED SUCCESSFULLY
=====
```

- Password Entropy: "LaboratoryPassword2026!" scored 150.76 bits — far exceeding the 50-bit minimum policy threshold ($22 \text{ chars} \times \log_2(94 \text{ charset}) \approx 6.854 \text{ bits/char}$).
- MD5 Hash (insecure): 096b1f381d46137f604347537db76776 — identical every single run for the same input. Attackers can precompute this value for all common passwords.
- PBKDF2 Run 1 salt: fcd7d5c65d34c46348a007bdae2a3278 — 16-byte CSPRNG random value. Hash: 3529fa3622127a42371c....
- PBKDF2 Run 2 salt: aaa275edf37f18f86af01e845afb6fec — DIFFERENT 16-byte CSPRNG salt. Hash:

29274da953a6e5ef83bd... — COMPLETELY DIFFERENT hash from the same password.

- This contrast directly demonstrates why salted KDFs defeat precomputed attacks: no two PBKDF2 hashes can be looked up against the same rainbow table.
- Failed login attempt #1 ("123456"): Denied — 2 attempts remaining before logout.
- Failed login attempt #2 ("admin123"): Denied — 1 attempt remaining before logout.
- Failed login attempt #3 ("wrong_pass"): Account locked! Exceeded max failed attempts (3).
- Correct password attempted AFTER lockout ("LaboratoryPassword2026!"): Still DENIED — "Account locked due to multiple failed attempts. Try again in 9 seconds." — confirms lockout state blocks all logins including correct credentials.

✓ **RESULT:** CLI engine executed successfully. Demonstrated: MD5 determinism vs PBKDF2 CSPRNG uniqueness, 150.76-bit password entropy, and account lockout triggered after 3 consecutive failures. Correct password blocked during active lockout window.

```
Windows PowerShell - Experiment 07 Interactive Authentication Testing

PS D:\Ethical Hacking Experiments\Ethical-Hacking-Experiments-2-and-7\Experiment-07> python scripts/app.py

=====
EXPERIMENT 07: PASSWORD SECURITY & AUTHENTICATION ENGINE
Course: MR23-1CS0432 - Ethical Hacking Laboratory (MRU)
=====

[1] Password Hashing & Salt Demonstration
Raw Input Password: LaboratoryPassword2026!
Password Entropy: 150.76 bits

[INSECURE] Unsalted MD5 Hash: 096b1f381d46137f604347537db76776

[SECURE] PBKDF2 (Run 1):
Salt 1 (Hex): fcd7d5c65d34c46348a007bdae2a3278
Hash 1 (Hex): 3529fa3622127a42371c35293515abfb1f8a9e26ba25df453549a5b7084b614d

[SECURE] PBKDF2 (Run 2 - Same Password, New CSPRNG Salt):
Salt 2 (Hex): aaa275edf37f18f86af01e845afb6fec
Hash 2 (Hex): 29274da953a6e5ef83bd4ed9626c24c61f1c5768d647cd70aa5df5c6ebc90c0b

[2] Registering Synthetic User 'student_admin'...
Registration Status: SUCCESS

[3] Simulating Failed Login Attacks & Account Lockout Defense:
Attempting '123456' -> [DENIED]: Invalid credentials. Attempts remaining before lockout: 2
Attempting 'admin123' -> [DENIED]: Invalid credentials. Attempts remaining before lockout: 1
Attempting 'wrong_pass' -> [DENIED]: Account locked! Exceeded max failed attempts (3).
Attempting 'LaboratoryPassword2026!' -> [DENIED]: Account locked due to multiple failed attempts. Try again in 9 seconds.

=====
LOCAL PRACTICAL EXECUTION TEST COMPLETED SUCCESSFULLY
=====

PS D:\Ethical Hacking Experiments\Ethical-Hacking-Experiments-2-and-7\Experiment-07> _
```

Figure 7.2: Interactive CLI authentication engine output (python scripts/app.py) showing MD5 hash, PBKDF2 salted hashes with unique CSPRNG salts, entropy measurement (150.76 bits), user registration, and account lockout after 3 failed attempts.

6. Security Analysis & Implications

The experiment demonstrates several fundamental password security properties with direct real-world implications for application developers, security engineers, and penetration testers.

Vulnerability / Weakness	Risk	Demonstrated Fix
Unsalted MD5 hash storage	CRITICAL — instant rainbow-table lookup; GPU can compute 10^9 hashes/sec	PBKDF2-HMAC-SHA256 with 16-byte CSPRNG salt and 100,000 iterations
No account lockout	HIGH — online brute-force allows unlimited guesses	Lockout after 3 failures; correct password blocked during window
Standard string comparison (==)	MEDIUM — timing attacks reveal partial hash matches	<code>hmac.compare_digest()</code> constant-time comparison in <code>auth_engine.py</code>
Weak password policies (low entropy)	HIGH — short/simple passwords cracked in seconds	Entropy calculation ($E = L \times \log_2(R)$) enforces minimum threshold
Same hash for same password across users	HIGH — one cracked hash reveals all users with same password	Unique CSPRNG salt per registration ensures unique stored hash per user

7. Countermeasures & Prevention

- Store passwords exclusively with a modern, slow key-derivation function — `bcrypt` (cost factor ≥ 12) or `Argon2id` (memory ≥ 64 MB, iterations ≥ 3 , parallelism ≥ 2) are the current best-practice recommendations per OWASP.
- For PBKDF2-HMAC-SHA256, use at least 600,000 iterations (NIST/OWASP 2023 recommendation). The experiment uses 100,000 — acceptable for demonstration, but production systems should use $\geq 600,000$.
- Generate all salts with a CSPRNG (Python's `secrets` module) — never the standard `random` module. Minimum salt length: 16 bytes (128 bits).
- Enforce a minimum password-entropy threshold (≥ 50 bits) at registration and provide real-time strength feedback to users.
- Implement account lockout with exponential back-off — lock after 3–5 failures, extending lockout duration for repeat offenders.
- Use constant-time hash comparison (`hmac.compare_digest()`) to prevent timing side-channel attacks on authentication.
- Transmit passwords exclusively over TLS (HTTPS) — plaintext credential transmission eliminates all password-storage protections.
- Integrate into a secure password reset flow — reset tokens must be single-use, time-limited, and generated by a CSPRNG.
- Monitor and alert on repeated authentication failures as an indicator of credential stuffing or brute-force campaigns.

8. Learning Outcomes

- Understood the evolutionary gap between insecure legacy password storage (plaintext, MD5) and modern key-derivation functions (PBKDF2, bcrypt, Argon2id), including the specific cryptographic reasons for each advancement.
- Observed through live code execution that identical passwords produce identical MD5 hashes (deterministic — enables precomputed attack) but completely different PBKDF2 hashes when CSPRNG salts are used (non-deterministic — defeats precomputed attack).
- Learned the mathematical definition of password entropy ($E = L \times \log_2(R)$) and its application in enforcing minimum password strength policies.
- Demonstrated account lockout rate-limiting as a defense against online brute-force attacks, observing that even the correct password is blocked during an active lockout window.
- Understood why constant-time comparison (`hmac.compare_digest`) is necessary to prevent timing side-channel attacks on hash verification.
- Gained practical experience writing and executing automated unit tests (Python unittest) as a formal software security verification methodology.
- Recognised the importance of CSPRNG-generated salts and why the standard random module is cryptographically unsuitable for security-sensitive operations.

9. Conclusion

Experiment 07 successfully demonstrated the principles of modern password security through live execution of a purpose-built Python authentication engine (`scripts/app.py` and `scripts/auth_engine.py`) on a Windows 11 system. All 4 automated unit tests (`test_entropy_calculation`, `test_salting_uniqueness`, `test_user_registration_and_authentication`, `test_account_lockout`) passed cleanly in 0.161 seconds, providing code-level verification of each security control. The interactive CLI session produced verifiable cryptographic evidence: an MD5 hash that is deterministic and identical across runs (096b1f381d46137f604347537db76776), contrasted against two PBKDF2-HMAC-SHA256 runs on the same password that produced entirely different salts (fcd7d5c6... and aaa275ed...) and hashes — directly demonstrating the security benefit of CSPRNG salting. Password entropy was measured at 150.76 bits for the test password, and account lockout was triggered and observed correctly after 3 consecutive failed authentication attempts. The experiment demonstrates that secure password storage requires a slow, salted key-derivation function, a minimum entropy policy, account lockout rate limiting, and constant-time comparison — none of which are provided by simple MD5 hashing.

10. Viva Voce Examination — Questions & Answers

Q1: Why is MD5 completely unsuitable for password hashing in modern systems?

Answer: MD5 is a fast cryptographic hash function capable of computing billions of hashes per second on consumer GPUs. It produces no salt, is vulnerable to precomputed rainbow-table attacks, and has known collision vulnerabilities. A stolen MD5 password hash database can be cracked almost instantly using freely available tools and lookup tables.

Q2: What is a cryptographic salt and why must it be unique per user?

Answer: A salt is a random sequence of bytes generated by a CSPRNG and stored alongside the password hash. A per-user unique salt ensures that two users with identical passwords (e.g., "password123") produce completely different stored hash values. This renders precomputed rainbow tables useless and forces attackers to brute-force each user's hash individually, even if they know the salt.

Q3: What is a Key Derivation Function (KDF) and how does it differ from a standard hash function?

Answer: A KDF (PBKDF2, bcrypt, Argon2) is intentionally designed to be computationally slow and resource-intensive through configurable iteration counts (PBKDF2) or memory-hard cost parameters (Argon2id). A standard hash (MD5, SHA-256) is designed for speed. The deliberate slowness of a KDF means that even if an attacker obtains the hash database, each password guess requires significant computation time.

Q4: What is the recommended minimum PBKDF2-HMAC-SHA256 iteration count per OWASP 2024?

Answer: OWASP recommends at least 600,000 iterations for PBKDF2-HMAC-SHA256, or 210,000 iterations for PBKDF2-HMAC-SHA512. The experiment uses 100,000 iterations for demonstration; production deployments should use the current OWASP recommendation, which increases over time as hardware improves.

Q5: Explain the difference between Argon2d, Argon2i, and Argon2id.

Answer: All three are variants of Argon2, winner of the 2015 Password Hashing Competition. Argon2d maximises resistance against GPU-based cracking (suitable for cryptocurrencies but vulnerable to side-channel attacks). Argon2i uses data-independent memory access patterns to resist side-channel attacks (suitable for hashing secrets on compromised systems). Argon2id is a hybrid combining both — recommended for general-purpose password hashing by OWASP and RFC 9106.

Q6: What is password entropy and how is it calculated?

Answer: Password entropy measures unpredictability in bits: $E = L \times \log_2(R)$, where L is password length (characters) and R is the character set size. For "LaboratoryPassword2026!" using all four character classes (lowercase, uppercase, digits, symbols — $R=94$), $E = 22 \times \log_2(94) \approx 150.76$ bits, well above the common 50-bit minimum policy threshold.

Q7: What is a timing attack in authentication systems and how does this experiment mitigate it?

Answer: A timing attack measures subtle differences in execution time of string comparison operations. A naive hash comparison (`computed_hash == stored_hash`) short-circuits as soon as a mismatch is found — an attacker can statistically measure which guesses take longer, implying more matching initial characters. Python's `hmac.compare_digest()` performs constant-time comparison regardless of where the mismatch occurs, making timing measurements useless.

Q8: What is credential stuffing and how does account lockout help against it?

Answer: Credential stuffing is an automated attack where stolen username/password pairs from one data breach are tested against unrelated services, exploiting password reuse. Account lockout limits the number of attempts per account per time window, slowing automated attacks from testing thousands of credentials per second to a negligible rate.

Q9: Why must salts be generated using the secrets module rather than the random module in Python?

Answer: Python's `random` module uses a Mersenne Twister PRNG, which is not cryptographically secure — its state can potentially be predicted from a sequence of outputs. The `secrets` module wraps the operating system's CSPRNG (`os.urandom()`), which provides cryptographically unpredictable output suitable for generating salts, tokens, and keys.

Q10: What is the security difference between storing a hash with and without a salt for a common password like "password123"?

Answer: Without a salt, "password123" always produces the same MD5 hash (482c811da5d5b4bc6d497ffa98491e38). An attacker can find this in precomputed tables instantly. With a unique CSPRNG salt, the same password produces a completely different hash value for every user and every registration event — an attacker must brute-force each hash individually, which with a proper KDF and high iteration count can take years per hash.